Waste MAS: многоагентная ИИ-система планирования вывоза ТБО в г. Астана

Nov 6, 2026 · @Nurda

1. Аннотация и введение

В этой работе построена многоагентная ИИ-система **Waste MAS**. Каждый день она решает, какие контейнеры с твёрдыми бытовыми отходами (ТБО) в Астане вывозить завтра и в каком порядке их объезжать. Работу выполняют оркестратор и шесть специализированных агентов: данные, прогноз, приоритеты, маршруты, проверяющий и отчёт. На демонстрационных данных (60 контейнеров, 21 день) система строит план из 2 рейсов для 9 контейнеров. Проверяющий агент утверждает план, а ни один агент не выполняет более 21,7% вызовов инструментов при лимите 40%. Прогноз заполненности с учётом календаря РК ошибается в среднем на 0,69 п.п. против 8,83 п.п. у наивного метода. Все 18 тестовых сценариев проходят.

Тема связана с магистерской работой «Разработка цифровой платформы для автоматизации сбора и вывоза мусора в городе Астана». Сегодня во многих городах контейнеры вывозят по фиксированному графику. Из-за этого одни площадки переполняются, а к другим машины едут за полупустыми контейнерами. Если на площадках стоят датчики заполненности, вывоз можно планировать по фактическим данным и прогнозу.

Цель работы: спроектировать и реализовать систему, которая по телеметрии контейнеров выдаёт проверенный план вывоза на следующий день и объясняет каждое решение.

Почему команда агентов, а не один агент

- **Разные компетенции.** Задача состоит из этапов с разной природой: валидация данных, статистический прогноз, правила отбора, комбинаторная оптимизация маршрутов, генерация текста. У каждого этапа свои данные, методы и критерий качества. Если отдать их одному агенту, получится монолит, в котором ошибку прогноза трудно отличить от ошибки маршрута.
- **Независимая проверка.** Агент, который построил план, не должен сам его утверждать. Отдельный VerifierAgent проверяет жёсткие ограничения и качество прогноза своими инструментами.
- **Переговоры при конфликте.** Когда рейсов не хватает, проверяющий отклоняет план с подсказкой `fix=defer`. Приоритизатор переносит наименее срочные контейнеры, маршрутизатор строит рейсы заново. Это настоящий цикл взаимодействия, а не фиксированный конвейер.
- **Сменяемость.** Любого агента можно заменить, например подключить дорожный граф вместо расстояний по прямой, не трогая остальных: контракт задаётся форматом сообщений и ключами в общей памяти.

2. Постановка задачи и данные

Вход системы — история заполненности контейнеров, выход — план на целевую дату: список контейнеров к вывозу, рейсы по машинам и объяснение каждого решения. План должен соблюдать три ограничения: вместимость машины, число доступных рейсов (машины × рейсы на машину) и обязательное обслуживание контейнеров, где ожидается переполнение.

Данные синтетические, потому что открытой телеметрии контейнеров Астаны нет. Генератор воспроизводим (seed = 42) и задаёт известную истинную модель, поэтому можно проверить, восстанавливают ли её агенты. Результаты относятся только к этому набору и не являются измерениями акимата.

Параметр	Значение
Контейнеры	60, по 12 в районах Есиль, Алматы, Сарыарка, Байконур, Нура
Период	04.10–24.10.2026, 21 день, 1260 измерений
Объём контейнера	0,66 / 1,1 / 2,5 м³ (веса 15 / 60 / 25%)
Темп накопления	4–10 п.п. в рабочий день, шум ±1,2 п.п.
Нерабочие дни	темп выше на 35% (люди дома)
Статус-кво	вывоз по фиксированному графику раз в 8–14 дней, остаток после вывоза 2–8%

В таком статус-кво 3,3% контейнеро-дней приходятся на заполнение 100%, а 37% из 113 вывозов делаются при заполнении ниже 60%. Это и есть проблема, которую решает система.

Календарь. Второй источник данных — нерабочие дни РК на 2026 год с учётом переносов (egov.kz). Для планирования важен День Республики: 25.10.2026 выпадает на воскресенье, и выходной переносится на понедельник 26.10. Последнее наблюдение в данных — суббота 24.10, поэтому прогноз на 1–2 дня попадает на длинные выходные.

Падение заполненности более чем на 15 п.п. считается вывозом: агент данных находит 113 таких событий.

3. Архитектура

Выбрана схема **«оркестратор + общая память (blackboard)»**. Оркестратор держит очередь задач и обменивается с агентами сообщениями `task` и `result`. Агенты не вызывают друг друга напрямую. Промежуточные результаты они пишут в SQLite через инструменты `db_read` и `db_write`, и каждая запись создаёт новую версию артефакта. Схема архитектуры приведена на рисунке 1.

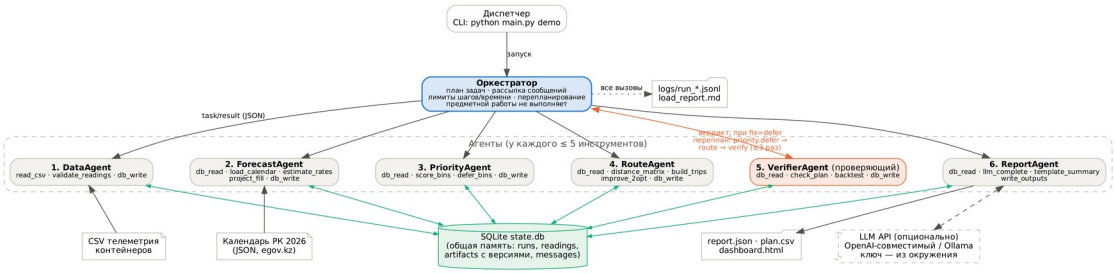


Рисунок 1. Архитектура Waste MAS (docs/architecture.png в репозитории).

Типовой план оркестратора: `data.ingest → forecast.forecast → priority.prioritize → route.plan_routes → verifier.verify → report.report`. Если проверяющий вернул `approved=false` и все нарушения помечены `fix=defer`, в начало очереди вставляется цепочка ремонта `priority.defer → route.plan_routes → verifier.verify`, не более 3 раз. Оркестратор смотрит только на поля `status`, `approved`, `fix` и `progress` в ответах. К телеметрии, прогнозам и маршрутам он не обращается, то есть предметной работы не выполняет.

Формат сообщения (dataclass `Message`, сериализуется в JSON, пишется в лог и в таблицу `messages`):

```
{"run_id": "replan", "step": 5, "sender": "verifier", "recipient":
"orchestrator",
 "type": "result", "ts": "2026-11-05T19:09:46.327+00:00",
 "payload": {"task": "verify", "status": "ok",
             "summary": {"approved": false,
                         "violations": [{"code": "fleet_limit", "trips": 3,
"available": 1, "fix": "defer"}]}}}
```

Таблица агентов

Агент	Роль	Вход → выход	Инструменты	Критерий завершения
Orchestrator	План задач, рассылка, перепланирование, лимиты	конфигурация, `result` → `task`, статус прогона	нет	очередь пуста, или сработал лимит, или ошибка
1. DataAgent	Загрузка и валидация телеметрии	путь к CSV → `readings`, `dataset_summary`	read_csv, validate_readings, db_write	все строки сохранены, или ошибка с номером строки
2. ForecastAgent	Прогноз с учётом календаря РК	`readings`, календарь → `forecasts`, `forecast_model`	db_read, load_calendar, estimate_rates, project_fill, db_write	прогноз для каждого контейнера

Агент	Роль	Вход → выход	Инструменты	Критерий завершения
3. PriorityAgent	Отбор, категории срочности, перенос	`forecasts` → `decisions`	db_read, score_bins, defer_bins, db_write	у каждого контейнера есть решение и причина
4. RouteAgent	Рейсы с учётом вместимости	`decisions` → `plan`	db_read, distance_matrix, build_trips, improve_2opt, db_write	каждый выбранный контейнер ровно в одном рейсе
5. VerifierAgent	Независимая проверка плана и прогноза	`decisions`, `plan`, `readings` → `verdict`	db_read, check_plan, backtest, db_write	вердикт и список нарушений с подсказкой `fix`
6. ReportAgent	Сводка диспетчеру и файлы	все артефакты → report.json, plan.csv, dashboard.html	db_read, llm_complete, template_summary, write_outputs	файлы созданы

У каждого агента не более 5 инструментов; это проверяется при создании агента (`MAX_TOOLS = 5`). Вызвать чужой инструмент агент не может. Задачи агентов не пересекаются: тест `test_agent_contracts` проверяет, что имена задач уникальны.

4. Алгоритмы агентов

ForecastAgent. Для каждого контейнера берутся приросты внутри цикла накопления, то есть между вывозами; дни с заполнением 100% исключаются. Сначала по однодневным приростам оценивается общий коэффициент нерабочих дней u: медиана по контейнерам отношения среднего прироста в выходной к среднему приросту в будний, минус 1. Затем для каждого контейнера считается базовый темп r: сумма приростов, делённая на сумму весов дней (рабочий день имеет вес 1, нерабочий — 1 + u). Прогноз на горизонт h дней:

$\hat{y}(T+h) = \min(100;\ y(T) + r \cdot \Sigma\ (1 + u \cdot [\text{день d нерабочий}]))$, где сумма берётся по дням d = T+1 … T+h.

На демо-данных агент оценил u = 0,36 при истинном значении генератора 0,35.

PriorityAgent. Контейнер выбирается к вывозу, если прогноз или текущее заполнение не ниже порога (80%). Категория `critical` назначается, если прогноз ≥ 100% (ожидается переполнение) или сейчас контейнер заполнен на 95% и больше. У каждого решения есть текстовая причина, например «прогноз 88,35% ≥ порога 80%». Задача `defer` переносит некритические контейнеры с наименьшим прогнозом, пока объём не станет не больше 90% вместимости парка. За один вызов переносится минимум один контейнер, так что цикл всегда продвигается. Критические контейнеры не переносятся никогда.

RouteAgent. Матрица расстояний считается по формуле Haversine (по прямой). Рейсы строятся методом ближайшего соседа с ограничением вместимости: машина едет к ближайшему контейнеру, который ещё помещается, а когда места не остаётся, возвращается в депо. Потом каждый рейс улучшается алгоритмом 2-opt, и рейсы распределяются по машинам по кругу. В типовом сценарии 2-opt сокращает путь с 53,34 до 49,89 км (−6,5%).

VerifierAgent. Проверяет жёсткие ограничения плана: каждый выбранный контейнер обслужен ровно один раз, лишних нет, все критические на месте, загрузка рейса не больше вместимости, дистанции корректны, и рейсов не больше, чем машины × рейсы на машину. К каждому нарушению прилагается подсказка `fix`: `defer`, если есть некритические контейнеры для переноса, или `null`, если исправить нечем. Отдельно агент делает бэктест: переобучает модель без последней даты и сравнивает её с наивным методом и линейным трендом.

ReportAgent. Собирает факты плана в JSON и просит LLM (любой OpenAI-совместимый `/chat/completions`, в том числе локальную Ollama) написать сводку для диспетчера. Защита от галлюцинаций `numbers_grounded` извлекает из ответа все числа и ищет их в фактах (допуск 0,1). Если числа нет в фактах, ответ отклоняется и используется детерминированный шаблон. LLM только объясняет решения и не принимает их.

5. Надёжность и безопасность

Каждый вызов инструмента проходит через `Runtime.call_tool`. Этот метод проверяет лимит времени, пишет событие в лог, повторяет вызов при временных ошибках и переводит остальные исключения в понятные сообщения. Другого пути к инструментам у агента нет, поэтому лог полный.

Требование ТЗ	Реализация	Сценарий
Лимит шагов	`max_steps = 20`, проверка перед каждой рассылкой; статус `aborted`	S12
Лимит времени	`timeout_s = 60`, проверка перед каждым шагом и вызовом инструмента	S13
Защита от зацикливания	не более 3 перепланирований; остановка, если `defer` ничего не перенёс или исправить нечем (`fix = null`); 2-opt ограничен 1000 итерациями	S05
Повторные попытки	2 повтора с экспоненциальной паузой (0,2 и 0,4 с) при сетевых ошибках, HTTP 429/5xx и блокировке SQLite; ошибки данных не повторяются	S16
Понятное сообщение при сбое	без трассировки стека, с номером строки и причиной; коды возврата 0 / 1 / 2	S06–S11

Требование ТЗ	Реализация	Сценарий
Логирование всех вызовов	JSONL: `run_start`, `plan`, `message`, `agent_call`, `tool_call`, `llm_call`, `retry`, `guard`, `fallback`, `replan`, `run_end`	все
Явное состояние	SQLite: таблицы `runs`, `readings`, `artifacts` (с версиями), `messages`	все
Ключи API не в коде	только переменные окружения и `.env` (в `.gitignore`); в лог пишется только `api_key_set`	тест `test_api_key_never_logged`

Выходной CSV защищён от инъекции формул: ячейки, которые начинаются с `= + - @`, экранируются. В HTML панели все пользовательские строки проходят через `html.escape`.

6. Эксперименты

6.1. Распределение нагрузки

В типовом сценарии (2 машины × 2 рейса × 8 м³, порог 80%) система делает 6 шагов и 23 вызова инструментов. Наибольшая доля — 21,7% — у ForecastAgent и RouteAgent. В прогоне с перепланированием (1 машина × 1 рейс, порог 70%) проверяющий отклоняет первый план из 3 рейсов. После переноса 8 контейнеров план утверждается: 9 шагов, 35 вызовов, максимум 28,6% у RouteAgent.



docs/logs/typical_load_report.md · replan_load_report.md · 23 и 35 вызовов

Равномерность заложена в архитектуру: инструменты укрупнены (один вызов `distance_matrix` на всю матрицу, а не по парам точек), и каждый агент делает от 3 до 5 вызовов за задачу. Оркестратор инструментов не вызывает. Даже при 3 перепланированиях доля RouteAgent составит 20 из 59 вызовов, то есть 34%, что ниже лимита.

6.2. Качество прогноза

Бэктест на контрольной дате 24.10.2026 (n = 53, 7 контейнеров исключено из-за вывоза или заполнения 100%) даёт MAE 0,69 п.п. у модели ForecastAgent. У линейного тренда текущего цикла MAE 3,10 п.п., у наивного метода «завтра = сегодня» — 8,83 п.п. Контрольная дата выпадает на субботу, поэтому выигрыш у тренда в основном даёт учёт календаря. На других seed (7 и 11) MAE модели составляет 0,67–0,70 п.п.

6.3. Тестовые сценарии

Из 18 сценариев проходят все 18 (`python main.py scenarios`, последний отчёт лежит в `docs/scenarios_report.md`). Дополнительно проходят 24 unit- и интеграционных теста. CI запускает их на Ubuntu (Python 3.11 и 3.13) и Windows (3.12).

ID	Сценарий	Ожидаемый результат	Факт
S01	Типовой сценарий	approved, 0 перепланирований, доля ≤ 40%	9 контейнеров, 2 рейса
S02	Горизонт 2 дня (перенос 26.10)	26.10 учтён, ≥ 10 контейнеров	12 контейнеров
S03	Качество прогноза	MAE модели < тренда < наивного	0,69 < 3,10 < 8,83
S04	Нехватка рейсов	перепланирование, критические обслужены	1 перепланирование, 8 перенесено
S05	Невыполнимый план (машина 3 м³)	rejected `fleet_limit`, без зацикливания	rejected после 1 попытки
S06	Контейнер больше машины	error с понятным текстом	error
S07–S11	Битые данные: 150%, нет файла, дубликат, заголовок, разные даты	error с причиной и номером строки	5 из 5
S12–S13	Лимит шагов = 3; лимит времени ≈ 0	aborted	aborted
S14	LLM работает	сводка от LLM, 1 вызов	сводка LLM
S15	LLM галлюцинирует	guard отклоняет, шаблон	число 1234 отклонено

ID	Сценарий	Ожидаемый результат	Факт
S16	LLM недоступна	≥ 2 повтора, шаблон	2 повтора, approved
S17–S18	Другой датасет (seed 7); порог 60%	approved, доля ≤ 40%	approved

7. Обсуждение архитектурных решений и ограничения

- **Оркестратор + blackboard в SQLite.** Состояние явное, переживает сбой и хранит версии плана. При прямых вызовах агент → агент не было бы единой точки контроля лимитов и логирования.
- **Решения принимают алгоритмы, LLM только пишет текст.** Результат воспроизводим, тесты стабильны, демонстрация работает офлайн. LLM-планировщик гибче, но может выдумать контейнер или нарушить вместимость.
- **Проверяющий — отдельный агент.** Он проверяет только инварианты и не зависит от того, как строился план. Проверка внутри RouteAgent сэкономила бы 4 вызова, но не ловила бы ошибки отбора.
- **Подсказка `fix` в вердикте.** Оркестратор направляет ремонт по коду и сам не решает, что переносить, — иначе он выполнял бы предметную работу.
- **Укрупнённые инструменты.** Каждый агент делает 3–5 вызовов, поэтому нагрузка ровная. С мелкими инструментами получился бы перекос в сторону RouteAgent.
- **Только стандартная библиотека Python.** Запуск одной командой на Python 3.11+. Фреймворки (LangGraph, CrewAI) дают готовые абстракции, но добавляют зависимости.

Ограничения.

- Данные синтетические. Хорошая оценка коэффициента выходных показывает, что метод корректен, но не подтверждает эффективность на реальной Астане.
- Расстояния считаются по прямой, без дорожного графа, пробок, временных окон и длины смены.
- Ближайший сосед с 2-opt — эвристика: она не гарантирует оптимум и не переставляет контейнеры между рейсами.
- Модель прогноза линейная, с одним общим коэффициентом для нерабочих дней; погода, сезонность и нарушения графика не учитываются.
- Бэктест использует одну контрольную дату; для выводов нужна скользящая проверка на нескольких неделях.
- LLM опциональна. Защита от галлюцинаций проверяет только числа, но не смысл фраз.

8. Заключение

Система Waste MAS закрывает требования Т3 для диапазона 90–100 баллов.

6 агентов с непересекающимися ролями (среди них агент-проверяющий) и оркестратор без предметной работы; у каждого агента не более 5 инструментов. Максимальная доля вызовов составляет 21,7% (28,6% при перепланировании) и подтверждена логами. Используются 4

внешних источника (CSV, календарь РК, SQLite, LLM API); есть лимиты, повторные попытки и полное логирование. Проходят 18 из 18 тестовых сценариев.

Дальнейшее развитие: подключить реальную телеметрию датчиков, заменить расстояния по прямой дорожным графом (OSRM), добавить временные окна и ограничение смены, проводить скользящий бэктест.

Как запустить: `cd research/waste-mas && python main.py demo`; тестовые сценарии — `python main.py scenarios`.

Источники

- Репозиторий проекта, модуль research/waste-mas
- egov.kz — Праздничные и выходные дни в РК в 2026 году
- Croes G. A. Method for solving traveling-salesman problems // Operations Research. 1958. Vol. 6, No. 6. — алгоритм 2-opt.
- Wooldridge M. An Introduction to MultiAgent Systems. 2nd ed. Wiley, 2009 — архитектуры с оркестратором и общей памятью (blackboard).
- OpenAI-совместимый Chat Completions API — интерфейс, через который подключается LLM (в том числе Ollama).